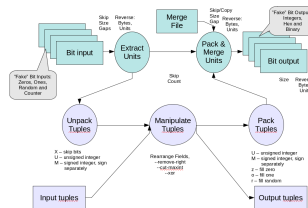


bdd — Bit Data Dump

High-performance CLI tool and Rust library to unpack, manipulate, stream, merge, and pack arbitrary-width bitstreams.

Originally written in Python 2, `bdd` was re-architected in modern Rust for speed, memory safety, and 20-year long-term maintainability with zero external C library dependencies.



Core Concepts: Arbitrary Bit Fields & Tuples

1. Handling Arbitrary-Size Bit Fields

Traditional binary utilities (such as standard `dd`, `hexdump`, or language byte buffers) operate strictly on byte (8-bit) or machine-word (16, 32, 64-bit) boundaries. In contrast, real-world data sources—such as embedded sensor telemetry, cryptographic streams, radio protocols, network packet headers, legacy binary archives, and compressed bitstreams—frequently pack data into unaligned bit fields of arbitrary widths (e.g., 1-bit flags, 3-bit status codes, 12-bit ADC samples, 57-bit mantissas, or 1024-bit cryptographic integers).

`bdd` handles bitstreams with single-bit precision through continuous stream buffering and arbitrary-precision integer arithmetic:

- **Continuous Bit Accumulator:** Input bytes are drawn into a streaming bit accumulator buffer (`BigUInt` buffer with bit tracking). Stream data is never forced into byte alignment; bits are shifted dynamically into the accumulator from the input source as required.
- **Arbitrary-Precision Extraction:** When an N -bit field is requested by a pattern, `bdd` checks whether at least N bits are available in the accumulator. If not, successive bytes are consumed from the source until sufficient bits are buffered. The top N bits are then extracted via bitwise shifting and masking:

$$\text{value} = (\text{buffer} \gg (\text{bits_in_buffer} - N)) \& ((1 \ll N) - 1)$$

Any remaining unconsumed bits stay in the accumulator for subsequent fields, preserving exact bit alignment across arbitrary field boundaries with zero bit loss.

- **Arbitrary Bignum Backing (`num-bigint`):** All integer fields are backed by arbitrary-precision `BigUInt` (unsigned) and `BigInt` (signed). Fields are never artificially clamped to 32 or 64 bits; they

can span hundreds or thousands of bits without risk of integer overflow.

- **Hardware-Accelerated Fast Path:** For fields ≤ 64 bits, `bdd` automatically selects native machine-register operations (such as direct CPU bit shifts and `u64::reverse_bits()`), ensuring raw I/O throughput is not throttled by bignum heap allocation overhead.

2. The Bit Field as a Manipulable Tuple

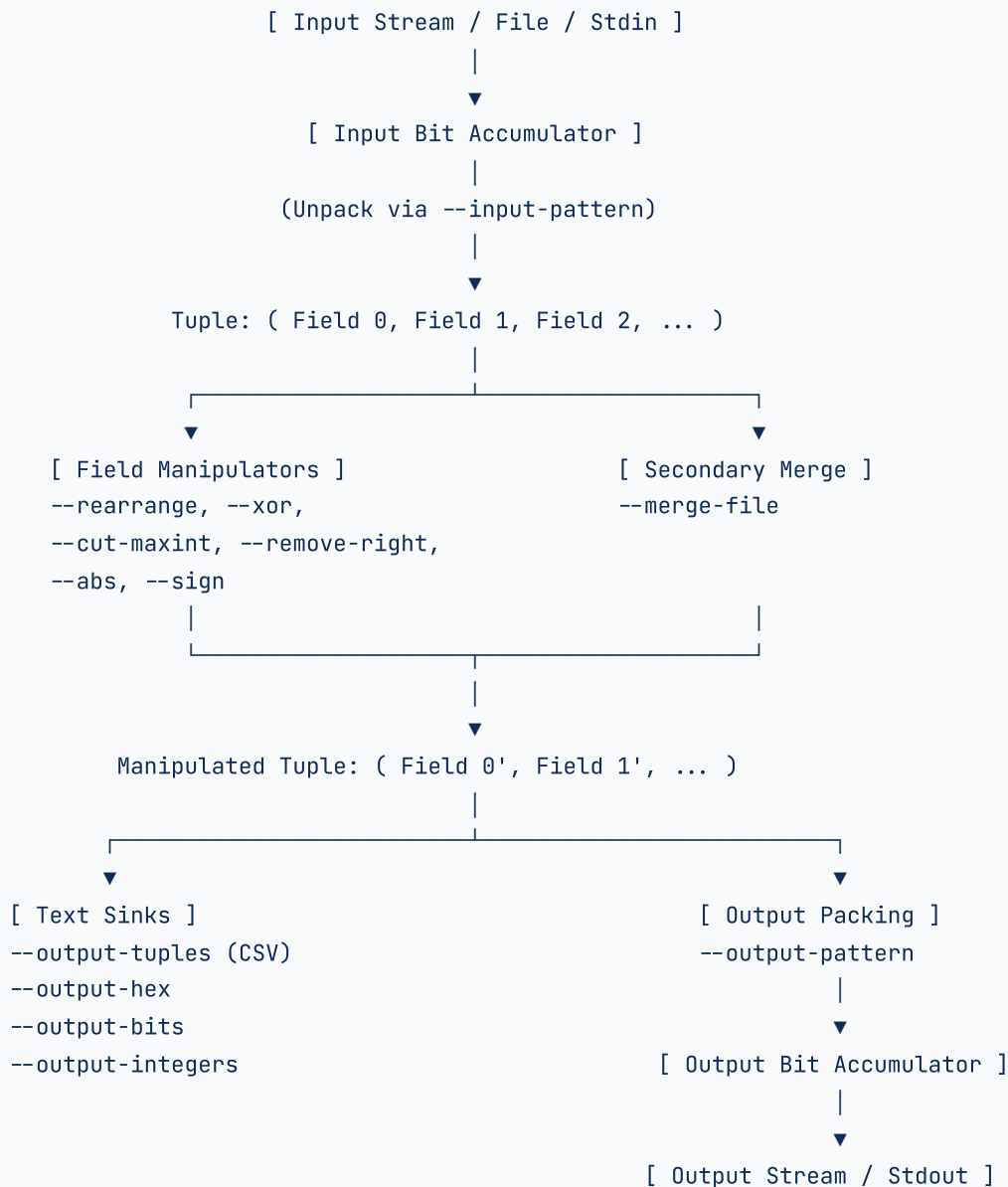
Once extracted, a unit of data is not treated as an opaque string of bits. Instead, **it is interpreted as a strongly-typed Tuple of discrete Fields** (analogous to a row in a relational database):

- **Input Pattern Decomposition:** When given an input pattern such as `--input-pattern=2U3U3U`, `bdd` unpacks each 8-bit chunk of the bitstream into a 3-element tuple:

```
Bitstream: [ 0 1 | 0 1 0 | 1 1 0 ]
Tuple:      ( Field 0: 2U = 1, Field 1: 3U = 2, Field 2: 3U = 6 )
```

Even a single unformatted unit such as `--input-unit=12` is internally a 1-element tuple: `( Field 0: 12U )`.

- **In-Flight Tuple Manipulation:** Because the stream is processed as structured tuples, manipulators can inspect, transform, reorder, or filter individual fields before repacking:
 - **Rearranging & Slicing (`--rearrange`):** Reorder, duplicate, or drop fields by index (e.g. `--rearrange=1,0` swaps field 0 and field 1; negative indices like `-1,0` access fields relative to the end of the tuple).
 - **Bit Truncation (`--remove-right`):** Strips the P least-significant bits from field F , useful for discarding low-order noise or downsampling sensor readings.
 - **Range Limiting (`--cut-maxint`):** Bounds field F to P bits (clamping values exceeding $2^P - 1$ to $2^P - 1$), useful for preventing overflow in downstream fixed-width systems.
 - **Bitwise Masking (`--xor`):** Applies a bitwise XOR key or inversion mask directly to a specific field in the tuple.
 - **Sign/Magnitude Decomposition (`--abs`, `--sign`):** Extracts absolute magnitude or isolates the sign bit (0 = positive, 1 = negative) from signed (`nS`) fields into dedicated tuple fields.
- **Stream Merging & Interleaving:** Additional files or secondary streams can be merged into the tuple stream (`--merge-file`), enabling multi-channel interleaving and address/timestamp tagging.
- **Packing & Sinks:** After manipulation, the resulting tuple is either:
 - **Repacked** into a new arbitrary bitstream according to an `--output-pattern` (where an output bit accumulator re-assembles fields into aligned bytes, injecting constant padding bits `nZ / n0` as specified).
 - **Serialized** directly to human-readable inspection formats: CSV tuples (`--output-tuples`), hex dump (`--output-hex`), bit-string dump (`--output-bits`), or raw integers (`--output-integers`).



Features

- **Arbitrary Bit-Widths:** Process bitfields of any length (not restricted to 8, 16, 32, or 64-bit boundaries).
- **Flexible Pattern Grammar:** Declarative input and output packing patterns supporting unsigned/signed integers, raw characters, floats, doubles, bit-reversed bytes, and padding constants.
- **Stream Generation & Merging:** Generate synthetic bit sequences (zeroes, ones, linear counters, PRNG noise) and interleave/merge multiple binary files.
- **Stream Transformations:** Rearrange field ordering, truncate bit ranges, clip integers to maximum bit bounds, execute bitwise XOR masks, and perform sign/magnitude conversions.
- **Multiple Output Formats:** Output raw binary bytes, hexadecimal representations, ASCII bit dumps (0 / 1), formatted integers, or CSV tuples.

- **Pure Safe Rust Core:** Clean library-first architecture (`src/lib.rs`) with typed error handling, no panics, and zero unsafe code.

Installation

Prerequisites

- Rust 1.70+ (`cargo` , `rustc`)

Build & Install

```
# Build optimized release binary
make release

# Or install directly to &approx;/.cargo/bin
make install
```

To build manually with Cargo:

```
cargo build --release
cp target/release/bdd &approx;/.local/bin/bdd
```

Pattern Grammar

Bit patterns specify the layout of fields within each input or output unit. A pattern consists of one or more field specifications, each written as `<bits><type>` :

SPECIFIER	DESCRIPTION	ALLOWED BIT WIDTHS
nU	Unsigned integer	Any positive integer (e.g. 2U , 12U , 128U)
nS	Signed integer (two's complement)	Any positive integer
nC	Character / byte string	Multiples of 8 bits (e.g. 8C , 32C)
nc	Character string with bit-reversed bytes	Multiples of 8 bits
32F	IEEE 754 single-precision float	Exactly 32 bits
64D	IEEE 754 double-precision float	Exactly 64 bits
nZ / nz	Constant zeroes (padding)	Output patterns only
n0 / no	Constant ones (padding)	Output patterns only

Units vs Patterns

- `--input-unit=N` is a shorthand for `--input-pattern=NU` .
- `--output-unit=N` is a shorthand for `--output-pattern=NU` .

Command Line Usage

Usage: bdd [OPTIONS] [FILE]

Arguments:

[FILE] Input file (defaults to standard input)

Options:

-p, --input-pattern <PATTERN>	Bit pattern to unpack input
-u, --input-unit <BITS>	Input unit size in bits (shorthand for <BITS>U)
-t, --input-tuples	Read fields from comma-separated input lines
-c, --input-counter	Generate sequential counter numbers
-0, --input-zeros	Generate endless stream of zero bits
-1, --input-ones	Generate endless stream of one bits
-r, --input-random	Generate pseudo-random bits
--skip-bits <BITS>	Skip initial bits before processing [default: 0]
--skip <UNITS>	Skip initial units before processing [default: 0]
--count <COUNT>	Process at most COUNT units (0 = infinite) [default: 0]
--reverse-input-bytes	Reverse bit order of each input byte
--reverse-input-units	Reverse bit order of each input unit
--output-pattern <PATTERN>	Bit pattern to pack output
--output-unit <BITS>	Output unit size in bits (shorthand for <BITS>U)
-x, --output-hex	Output units as hexadecimal strings
-b, --output-bit	Output units as ASCII bit strings ('0' and '1')
-i, --output-integer	Output unsigned integer representation per unit
-T, --output-tuples	Output fields as comma-separated tuples
--reverse-output-bytes	Reverse bit order of each output byte
--reverse-output-units	Reverse bit order of each output unit
--rearrange <FIELDS>	Reorder output fields by index (e.g. "1,0" or "-1,0")
--cut-maxint <F,P>	Clip field F to fit in P bits (F:field index, P:bit width)
--remove-right <F,P>	Remove P least-significant bits from field F
--xor <F,P>	Bitwise XOR field F with value P
--abs <FIELD>	Convert signed field to absolute value
--sign <FIELD>	Extract sign bit (0 = positive, 1 = negative)
--merge-file <PATH>	Interleave stream from another file
-h, --help	Print help
-V, --version	Print version

Examples

1. Simple Bit Inspection & Hex Dumps

Extract individual bits from standard input and display them:

```
echo -n "A" | bdd --input-unit=8 --output-bits
# Outputs: 01000001
```

Print hexadecimal values of 12-bit unpacked units:

```
bdd --input-unit=12 --output-hex < input.bin
```

2. Endianness & Byte Swapping

Swap every pair of adjacent bytes in a binary stream:

```
bdd --input-pattern=8U8U --rearrange=1,0 --output-pattern=8U8U < input.bin > swapped.bin
```

Reverse bit order within every byte:

```
bdd --reverse-input-bytes < input.bin > bits_reversed.bin
```

3. Bitfield Packing and Unpacking

Pack non-byte-aligned numbers into arbitrary bit containers:

```
# Combine 2-bit, 3-bit, and 3-bit fields into single 8-bit octets:
printf "1,2,3\n3,7,7\n" | bdd --input-tuples --output-pattern='2U3U3U' | od -t o1
# Outputs: 123 377
```

Unpack 10-bit ADC sensor data into comma-separated tuples:

```
bdd --input-pattern=10U --output-tuples < sensor.raw
```

4. Synthetic Stream Generation

Generate 16 sequential bytes formatted as hex:

```
bdd --input-counter --count=16 --output-hex
# Outputs: 00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f
```

Generate 1 megabyte of zero padding:

```
bdd --input-zeros --count=$((1024 * 1024)) --output-unit=8 > 1MB_zeros.bin
```

5. Stream Merging & Interleaving

Interleave a sequential index address with data bytes from `/etc/passwd`:

```
bdd --input-counter --count=16 --input-unit=12 --output-unit=12 --merge-file=/etc/passwd --output-hex
```

Architecture & Codebase Design

The codebase is engineered defensively for decades-long maintainability:

```
src/
├─ lib.rs          # Public library crate interface
├─ main.rs         # Lean executable entry point & exit code translation
├─ error.rs        # Strongly-typed BddError hierarchy
├─ field.rs        # Arbitrary-precision Field enum & hardware-accelerated bit-reversals
├─ pattern.rs      # Grammar parser, TupleUnpacker, and TuplePacker
├─ stream.rs       # Stream generators (File, Counter, Zeros, Ones, Random, Tuples)
├─ sink.rs         # Output writers (Binary, Hex, Bit, Integer, CSV Tuples)
├─ manipulator.rs  # Field transformations (Rearrange, Xor, CutMaxint, Abs, Sign)
├─ cli.rs          # Clap CLI definition & mutual-exclusion configuration validation
└─ engine.rs       # End-to-end pipeline execution orchestrator
```

Key Design Principles

1. **Strict Type Safety:** All errors flow through `BddError`. Functions return `Result<T, BddError>` instead of panicking or calling `std::process::exit`.
2. **Fast-Path Bit Reversal:** Sub-64-bit integer bit reversals execute via direct hardware `u64::reverse_bits()`, falling back to `BigUint` bit arithmetic only when necessary.
3. **Byte-Level String Integrity:** The `C` and `c` pattern types store raw bytes internally (`Field::Bytes`) rather than lossy UTF-8 conversions, guaranteeing bit-perfect roundtrips.
4. **Automated Verification:** Integrated test runner runs both native Rust unit tests and legacy golden-file integration tests.

Development & Testing

Run the test suite:

```
make test
# or: cargo test --all-targets --all-features
```

Run benchmarks:

```
make bench
# or: cargo bench
```

Run code linter and formatting checks:

```
make lint
make check-fmt
```

Performance & Throughput Benchmarks

`bdd` achieves high throughput across arbitrary bit boundaries, balancing hardware register acceleration for sub-64-bit units with arbitrary-precision arithmetic for large bignum fields.

Measured via `make bench` (`benches/throughput.rs`) on Linux x86_64:

OPERATION	TOTAL VOLUME	THROUGHPUT (BITS/S)	THROUGHPUT (BYTES/S)	NOTES
Hardware 64-bit Bit Reversal	640 Mbits	14.42 Gbps	1,803 MB/s	Direct CPU <code>u64::reverse_bits()</code>
Bignum 1024-bit Packing/Unpacking	25.6 Mbits	6.49 Gbps	811.5 MB/s	Large-block bignum bitfield packing
Bignum 256-bit Packing/Unpacking	25.6 Mbits	1.73 Gbps	216.1 MB/s	SHA-256 size field packing/unpacking
Synthetic 8-bit Linear Stream	80 Mbits	270.1 Mbps	33.8 MB/s	Continuous bit generation & sink flush
Bignum 1024-bit Bit Reversal	25.6 Mbits	43.1 Mbps	5.4 MB/s	Full arbitrary-precision bit reversal
1-bit Single-Bit Resolution Stream	2.0 Mbits	39.2 Mbps	4.9 MB/s	Single-bit slice accumulation & packing
Unaligned 3-bit to 8-bit Extraction	9.0 Mbits	39.1 Mbps	4.9 MB/s	Cross-byte boundary accumulation
Tuple Pipeline (<code>2U3U3U</code> → Rearrange)	8.0 Mbits	33.9 Mbps	4.2 MB/s	Multi-field unpack, reorder & repack

Documentation & PDF Generation

All project documentation compiles into clean, print-ready vector PDF and HTML files:

- [README.pdf](#) : Generated from `README.md` via headless Chromium / Puppeteer with vector math, diagrams, and GFM styling.
- [docs/bdd.1.pdf](#) : Unix manual page rendered as clean vector PDF via `groff` and `ps2pdf` .
- [docs/bdd.1.html](#) : Unix manual page rendered as standalone HTML.
- [docs/Presentation.pdf](#) : Architectural slide deck compiled from OpenDocument Presentation (`legacy/old_src/Presentation.odp`) via headless LibreOffice.

To regenerate all documentation artifacts in one command:

```
make docs
```

License

GPL-3.0-or-later. Original author: Esa Turtiainen.